



FOR JAVA DEVELOPERS MOVING TO NESTJS

French customer. German receipt.

Spring Boot to NestJS, episode 39: email and translations

Email and translations both have fallback policies you can miss in development.

Spring's `MessageSource` falls back to the server's locale by default: a French request on a German server got German.

The new first-party `@nestjs/i18n` fell back to its default locale, and returned the raw key for a key in no catalog.

A silent mail server blocked Spring past 60 s with the timeouts at their defaults; an unobserved send failure ended a Nest process.

In scope

- `MessageSource` and `@nestjs/i18n`
- Locale fallback and missing keys
- SMTP timeouts
- Send and forget

Not in scope

- Email templates
- Pluralisation and ICU formats

At the time of recording `@nestjs/i18n` is 0.0.1 and needs Nest 12.1.

01 The measurements

SPRING, fr, GERMAN JVM LOCALE	German
SPRING, fallback-to-system-locale=false	English
NEST, fr	English (defaultLocale)
A KEY IN NO CATALOG	Spring 500; Nest 200 + the raw key
SILENT SMTP, DEFAULTS	Spring past 60 s; nodemailer 30 s
SPRING, TIMEOUTS 5000 ms	failed after 5 s
SEND AND FORGET, REFUSED	Spring serving; Nest exit code 1

Catalogs en (default) and de.

02 Spring: decide the fallback, set the timeouts

Stop the fallback to the server's locale, and set every SMTP timeout.

```
application.properties

spring.messages.fallback-to-system-locale=false
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000
```

fallback off: English; timeouts 5000: failed after 5 s

Run on a real machine, 2026-10-01, by scripts/verify-i18n-mail.sh in the course repository.

03 Nest: choose the policy, observe every send

Pick a missing-key policy you can live with in front of a customer, and never leave a send promise unobserved.

```
app.module.ts

I18nModule.forRoot({
  loader: new JsonI18nLoader({ path: join(import.meta.dirname, "i18n") }),
  defaultLocale: 'en',
  missingKey: 'throw', // or 'fallback' (the default): the raw key
}),
```

```
receipts.service.ts

this.transport.sendMail(receipt).catch((err) => this.logger.error(err));
```

unobserved: HTTP 200, then the process exited with code 1

Run on a real machine, 2026-10-01, by scripts/verify-i18n-mail.sh in the course repository.

Before a message reaches a customer

1. What does an unsupported locale get?
2. What does a missing key become?
3. How long can a send block a request?
4. Who observes a failed send?
5. Does a 200 mean the email was sent, or only accepted?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-i18n-mail.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS i18n https://docs.nestjs.com/application/i18n

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.